

Lab 7

Write a program to solve the following water jug problem using the approach of constraint satisfaction problem.

Example problem

Problem Definition:

- Jug A Capacity = 5 liters
- Jug B Capacity = 3 liters
- Initial State = (0, 0)
- Goal State = Jug A contains 1 liter $\rightarrow (1, _)$

Constraints:

- Jug A can hold max 5 liters: $0 \leq a \leq 5$
- Jug B can hold max 3 liters: $0 \leq b \leq 3$
- Valid operations:
 - Fill either jug to its max
 - Empty either jug
 - Pour from one jug to another without spilling

Pseudocode

Function is_goal(state):

Return True if state.A == 1

Function get_next_states(state):

Let (a, b) = state

max_a = 5

max_b = 3

Initialize empty list: successors

Constraint-Based Actions

Add (5, b) // Fill Jug A

Add (a, 3) // Fill Jug B

Add (0, b) // Empty Jug A
Add (a, 0) // Empty Jug B
pour = min(a, max_b - b)
Add (a - pour, b + pour) // Pour A → B

pour = min(b, max_a - a)
Add (a + pour, b - pour) // Pour B → A

Return successors

Function water_jug_bfs(start_state):
Initialize visited_set = empty set
Initialize queue with (start_state, path_list)

While queue is not empty:
Pop (current_state, path) from queue

If is_goal(current_state):
Return path

For each next_state in get_next_states(current_state):
If next_state not in visited_set:
Add next_state to visited_set
Enqueue (next_state, path + [next_state])

Return None // If no path found

Main:
Call water_jug_bfs(start_state = (0, 0))
If solution found:
Print step-by-step jug contents to goal state
Else:
Print "No solution found"

Report format

Title: Write a program

Date: Lab assigned date

Objective

- To implement the water jug problem solution using constraint satisfaction problem approach

Theory

- Constraint satisfaction problem
 - Water jug problem

Implementation

- Write down the problem statement
- Python Code
- Output (Screenshots of your output)

Conclusion